

ScaleDown: Learning Queue-Aware Resource Requests for Carbon-Efficient Training under Completion Budgets

Anonymous submission

Abstract

An ordinary user of a shared GPU cluster controls resource requests, while the batch scheduler controls when they run. This separation makes carbon-aware training a sequential decision problem: node count changes training efficiency, queue wait, and the carbon intensity encountered during execution. We present ScaleDown, a controller that selects the node count of each checkpointed training chunk under a slider-selected completion budget. Its policy combines remaining work and time with public queue history and a causal carbon-intensity forecast sequence, without requiring an admission-time predictor. An undiscounted, constrained actor-critic objective accounts for variable chunk durations and deadline misses. To handle unmeasured node power, we factor carbon accounting into execution exposure and an explicit power scenario. Within a three-factor power model, two endpoint evaluations characterize the full sensitivity interval, enabling one policy to optimize across that interval. The evaluation compares against all fixed scales, a fixed-policy mixture, and a queue-aware planning baseline on two trace-derived demand scenarios with published AMSP throughput profiles [E1: **scenario and input validation: pending**]; [E2: **baseline-relative carbon and deadline results: pending**]; [E3–E4: **mechanism and robustness results: pending**]. The resulting claims concern modeled job-attributed operational carbon; facility emissions and production deployment are outside the evaluation.

1 Introduction

The operational carbon associated with training depends on electricity consumption and when that electricity is used (Patterson et al. 2021). Carbon-aware systems exploit temporal, spatial, and allocation flexibility (Hanafy et al. 2023; Sukprasert et al. 2024). On a shared batch cluster, however, a user controls a *request*, while Slurm determines admission, placement, and start time. A resource choice may take effect hours later, when the carbon intensity differs from that observed at submission.

Node count couples three consequences. Fewer nodes can reduce node-hours under sublinear strong scaling, but lengthen execution and increase the number of queue admissions needed by a long run. A smaller request may start

sooner, yet overlap a less favorable carbon interval. Near completion, a faster final chunk may avoid another queue wait. The useful decision therefore depends on remaining work, available time, and the execution window of each request.

The central question is *when adapting requests provides value beyond a good fixed scale*. With constant carbon intensity, no waiting, and no chunk overhead, we show that mixed-scale execution lies in the convex hull of fixed-scale time-carbon points. Scaling inefficiency alone offers no gain below that frontier. Under deadlines, a preplanned within-run mix can still outperform a per-run randomized fixed policy without new queue observations. We therefore retain both fixed mixtures and a plan chosen once at submission when testing the value of online adaptation.

ScaleDown exposes a completion-budget slider and makes repeated node-count decisions without modifying Slurm. Its main actor-critic policy uses visible queue history, remaining work and time, action execution descriptors, and a causal CI forecast sequence. It requires no explicit wait predictor: continuation costs are learned from realized replay outcomes, and each checkpoint reveals how much budget remains. Hidden admissions still limit predictability and any deadline claim. We compare against explicit rollout planning and a matched learned policy that commits its full sequence before submission, separating dynamic feedback from the benefit of a preplanned mix.

Complete-node power is unavailable, so we separate execution evidence from carbon estimates. The power model has three factors: a normalization reference, an unknown workload coefficient, and a scale coefficient. For a fixed execution log, carbon is affine in the scale-model parameter. Two endpoint comparisons thus characterize robustness within the declared interval, without determining absolute power or repeating scheduler simulations. We use this property to train against both endpoint objectives and report where comparative conclusions change.

Our evaluation uses published AMSP performance profiles and two historical arrival streams to define controlled contention scenarios. Hardware labels do not determine admission once resource demands and scheduling rules are fixed; this abstraction does not predict how old jobs would run on new hardware.

The paper makes three contributions:

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

- A completion-budget formulation and fixed-scale null case that separate scaling efficiency, preplanned mixing, and online adaptive value.
- A user-level controller combining realized budget feedback, public queue history, and undiscounted constrained learning without an admission predictor.
- An exposure-based sensitivity analysis and a matched evaluation against fixed, plan-once, and replanning baselines.

2 Related Work

Carbon-aware elasticity. CarbonScaler varies a batch workload’s cloud allocation with carbon intensity and uses a greedy marginal-resource allocation algorithm (Hanafi et al. 2023). It already establishes resource elasticity as a carbon-control mechanism. ScaleDown addresses the additional delay between requesting resources and obtaining them, as well as repeated admission for checkpointed chunks. A direct transplant of cloud autoscaling would change the available controls; our controls retain repeated admissions and charge every queue wait. Studies of temporal and spatial shifting also show that flexibility and overhead constrain attainable carbon savings (Sukprasert et al. 2024). We report the scope of the opportunity rather than assume that lower-carbon timing is always available.

User-side provisioning. QBETS estimates wait bounds from historical time series (Nurmi, Brevik, and Wolski 2008); our main policy learns completed-run costs without an admission estimator. Mirage uses learning to provision follow-up resources for low-interruption services on batch GPU clusters (Ding et al. 2023). We use its trace-replay infrastructure, but optimize the node count of a ready training chunk under a completion budget. Scheduler-level methods such as DRAS act on the scheduling policy itself (Fan et al. 2021). Our policy has no such authority and cannot observe hidden priority or reservation state.

Constrained learning and accounting uncertainty. Constrained policy optimization separates a resource objective from a service constraint (Achiam et al. 2017); PPO supplies a practical actor–critic optimizer (Schulman et al. 2017). We use these established ideas, without claiming a new general-purpose RL optimizer or a constraint-satisfaction theorem. The method contribution is the request-conditioned decision representation and its coupling to a tractable power-scenario objective. Unlike operational-carbon measurement, the latter assesses consequences conditional on a model. An algebraic sensitivity certificate does not validate the model’s physical parameters.

3 Problem and Adaptive Opportunity

Control and work. A target run consists of U_* prescribed optimizer updates, executed as sequential checkpointed chunks. At boundary k , time is t_k and remaining work is U_k . The action is a feasible node count $n_k \in \mathcal{A} = \{4, 16, 64, 128\}$. The next chunk is submitted immediately; there is no intentional deferral, concurrent reservation, or resizing of a running chunk. Scale choices may rise or fall

between allocations; “down means using fewer than the maximum supported nodes, not irreversible shrinking. We use $H = 48$ hours as the study’s maximum requested wall time, subject to the chosen partition’s limits; it is not a universal Slurm limit.

For sharded or replicated data parallelism without tensor or pipeline parallelism, let g be GPUs per node ($g = 8$ for the published AMSP profiles), b the per-GPU micro-batch, and a_n accumulation. Each action satisfies

$$B_* = ngba_n, \quad a_n = B_*/(ngb) \in \mathbb{N}. \quad (1)$$

The same data order, sequence length, optimizer-update target, and update-indexed learning-rate schedule apply across actions. Accumulation implements the fixed batch (Shoeybi et al. 2019); it does not guarantee bitwise equality or equal final quality. A configuration that fails the batch, memory, or partition constraints is masked for every method.

Published steady-state throughput determines q_n , the updates per hour. Initialization/restart r_n and complete-checkpoint time h_n are declared scenarios, not measurements supplied by the throughput source. Define

$$\begin{aligned} v_{k,n} &= \min\{U_k, \lfloor q_n(H - r_n - h_n) \rfloor\}, \\ d_{k,n} &= r_n + v_{k,n}/q_n + h_n, \\ U_{k+1} &= U_k - v_{k,n_k}, \quad t_{k+1} = t_k + w_k + d_{k,n_k}. \end{aligned} \quad (2)$$

Only positive-progress actions are allowed. The request’s wall time is $d_{k,n}$ rounded up to the scheduler’s time granularity, capped at H ; the job releases resources on completion. Thus the final partial chunk has a shorter request, which can change admission. The assumed restart contract includes model, optimizer, learning-rate state, global update index, random-state metadata, and the next global sample position; this study does not implement or validate AMSP state migration. The final output save is included in h_n . End-to-end turnaround time is $T = t_K - t_0$, including every queue wait and allocation interval.

A fixed-scale null case. The following observation defines what scaling profiles alone can establish.

Proposition 1 (No improvement below the fixed hull)

Suppose update rates and per-node powers are constant for each scale, carbon intensity is constant, and there are no waits or restart/checkpoint overheads. Every execution of the same work, including one that switches scales, has a time–carbon pair in the convex hull of the fixed-scale pairs.

Proof. Let x_n be the fraction of updates executed at scale n , so $x_n \geq 0$ and $\sum_n x_n = 1$. If $T_n = U_*/q_n$ and C_n are the fixed-scale totals, additivity gives $(T, C) = \sum_n x_n(T_n, C_n)$. $\square$

This is a statement about mean time–carbon geometry, not a per-job deadline guarantee for randomized fixed policies. A preplanned sequence can complete individual runs at an intermediate time–carbon point even when a per-run fixed mixture violates a deadline. Thus beating a feasible fixed mixture alone does not prove that queue feedback is useful. Under real queueing, a request changes both its current wait and the time and scheduler state at which the next request arrives; the linear decomposition need no longer hold. Likewise, time-varying carbon intensity makes an update’s

carbon depend on its execution window. These effects create an opportunity, not a guarantee of an adaptive gain. Our main comparison retains all fixed scales, a deployable mixture, and a plan-once controller; action-switch frequency alone is not evidence of improvement.

What the slider can establish. Let $V_{\mathcal{P}}(D)$ be the infimum of the endpoint objective in Equation 8 over feasible policies in class $\mathcal{P}$. These classes describe admissible controllers with history, not the expressiveness of a particular network. Fixed policies and their per-run mixtures are contained in pre-planned sequences, which are contained in feedback policies. Therefore $V_{\text{feedback}}(D) \leq V_{\text{preplanned}}(D) \leq V_{\text{fixed}}(D)$. Each ideal envelope is nonincreasing in D , since relaxing the budget retains every formerly feasible policy. This is an opportunity bound, not a guarantee that finite PPO training finds the envelope. Strict improvement requires useful timing or admission variation; the null case above rules out claiming it from scaling inefficiency alone. A sequence that beats both fixed points still needs a preplanned comparator to establish the value of feedback.

4 Carbon Exposure with Unmeasured Power

Fix one machine–workload pair; its indices are suppressed below. The mean complete-node power assigned to scale n is

$$\hat{P}_n = P_0 \kappa s_n, \quad s_n = \rho + (1 - \rho) \eta_n, \quad \eta_n = \frac{W_4}{W_n}, \quad (3)$$

where $W_n = nU_{\text{profile}}/q_n$, $\kappa > 0$, $s_4 = 1$, and $0 \leq \rho \leq 1$. The reference $P_0 = 1 \text{ kW}$ fixes a normalization unit; κ absorbs the unknown workload-specific node-power level. Neither is a reported A800 server measurement. We report within-panel carbon ratios and node-hours, rather than absolute carbon or specification-based node-power estimates. The workload coefficient κ sets the four-node power level. The scale factor interpolates between constant per-node power ($\rho = 1$) and a constant-useful-work dynamic-energy limit ($\rho = 0$, without overhead). η_n is a scaling-efficiency ratio, not measured utilization, and ρ is not the CPU fraction of node power. We use this family as a sensitivity model; its parameter interval is a declared scenario, not a confidence interval. At constant CI and without overhead, Equation 3 gives total energy $P_0 \kappa [\rho W_\pi + (1 - \rho) W_4^*]$, where W_π is the execution’s node-hours and W_4^* is the cost of doing all target updates at four nodes. For $\rho > 0$, changing this parameter cannot reverse an energy ranking by node-hours. It changes the magnitude of savings in this limit, not which execution uses less energy.

For an execution log π , let $\mathcal{I}_{\pi,n}$ contain all intervals allocated to the target at scale n , including restart and checkpoint. With realized average grid intensity $c(t)$, define the scale-resolved exposure

$$L_{\pi,n} = n \int_{\mathcal{I}_{\pi,n}} c(t) dt, \quad C_\pi(\rho) = P_0 \kappa \sum_n s_n L_{\pi,n}. \quad (4)$$

Power is in kW and time in hours. For the primary EIA scenario, intensity is in kgCO_2/kWh ; absolute C_π remains unknown without κ . Non- CO_2 greenhouse gases are excluded.

Replacing $c(t)$ with 1 yields IT energy; summing allocated node time yields node-hours without a power assumption. Applying the same mean $\hat{P}_n$ to all allocated phases is an explicit approximation. Phase durations are retained in the log so its effect can be examined without claiming phase-specific power measurements.

Writing $A_\pi = \sum_n L_{\pi,n}$ and $B_\pi = \sum_n \eta_n L_{\pi,n}$ gives

$$C_\pi(\rho) = P_0 \kappa [B_\pi + \rho(A_\pi - B_\pi)]. \quad (5)$$

Proposition 2 (Endpoint comparison) *For fixed policies and execution logs, $C_\pi(\rho) \leq C_b(\rho)$ for every $\rho \in [\rho_-, \rho_+]$ if and only if this inequality holds at both endpoints. The same statement holds for expected carbon under a fixed distribution of arrivals and policy randomness.*

Proof. The difference is affine in ρ by Equation 5; its maximum on a closed interval occurs at an endpoint. Expectation preserves the affine form. $\square$

The common positive factor $P_0 \kappa$ cancels from within-panel ratios and rankings. No workload coefficient sweep is needed for these comparisons, although unknown coefficients preclude pooling absolute carbon across workloads. For a positive affine reference $C_{\text{ref}}(\rho)$, the ratio $\mathbb{E}[C_\pi(\rho)]/C_{\text{ref}}(\rho)$ is linear-fractional and has a derivative of constant sign (or zero). Its worst value also occurs at an endpoint. Policies must remain fixed while computing this result: retraining at each ρ would evaluate different policies, not certify one policy.

For powers outside this family, Equation 4 remains useful. If each scale exposure is no greater than a baseline’s, dominance holds for all positive scale-power coefficients. Otherwise a reversal is possible, and the result must retain a power-model qualification. A more informative local test allows $s_n = s_n^0(1 + \delta_n)$ for $n \neq 4$, with $|\delta_n| \leq d < 1$ and $s_4 = 1$. Writing $\Delta L_n = L_{\pi,n} - L_{b,n}$, the worst normalized difference is exactly

$$\max_{\delta} \frac{C_\pi - C_b}{P_0 \kappa} = \sum_n s_n^0 \Delta L_n + d \sum_{n \neq 4} s_n^0 |\Delta L_n|. \quad (6)$$

This tests independent scale-power errors using the same logs, without adding a power-model layer. Queueing draws no allocated energy at this accounting boundary. We exclude embodied emissions, idle and background-job power, and shared facility overhead. The estimated quantity is job-attributed operational carbon, not avoided grid emissions.

5 ScaleDown Controller

5.1 A Budget and a Causal Observation

A slider $s \in [0, 1]$ selects a completion budget $D(s) = T_{\text{ref}}[\beta_{\min} + s(\beta_{\max} - \beta_{\min})]$. Its left end requests a tighter finish; its right end permits more time to reduce carbon. The supported ticks are frozen before evaluation; intermediate positions require validation, not curve interpolation. The miss tolerance ϵ and power interval are experiment settings, not additional sliders. The chosen D is fixed for the run. At boundary k , actual remaining time is $b_k = D - (t_k - t_0)$, which may be negative. The policy observes

$$o_k = [U_k/U_\star, b_k/T_{\text{ref}}, D/T_{\text{ref}}, z_k, g_k, \{\phi_{k,n} : n \in \mathcal{A}\}], \quad (7)$$

where $T_{\text{ref}} > 0$ is a frozen training-set time reference and z_k summarizes visible queue history and calendar time. Each $\phi_{k,n}$ contains node fraction, normalized rate q_n , planned work $v_{k,n}/U_*$, allocation duration $d_{k,n}$, requested limit, and efficiency η_n . These descriptors include the shorter final chunk. The vector g_k contains 28 consecutive six-hour averages of a causal CI forecast, starting at t_k , normalized by a positive training reference. A rolling past hour-of-week forecast supplies this one-week horizon; the controller is not given a predicted start time.

Decisions without an admission predictor. The main actor and critic consume neither predicted waits nor exposure integrated over a predicted execution window. They learn continuation costs from complete replay outcomes in one end-to-end training procedure. No supervised admission model precedes policy training. After each completed allocation, actual elapsed time updates b_k and completed work updates U_k before the next decision. Only physical work feasibility masks actions; a predicted deadline cannot exclude a request. This removes an explicit wait-calibration prerequisite, but the learned policy still depends on relationships present in its training environment. It neither eliminates uncertainty nor establishes transfer to unseen scheduling rules. Feedback cannot recover a wait already incurred.

The queue history uses counts, node demand, requested limits, elapsed waits, occupied nodes, and elapsed running time at fixed lags. It never uses a running job’s eventual completion time. Missing fields have explicit masks; site visibility must be checked rather than inferred from trace completeness. The observation is partially observed: hidden Slurm state need not be recoverable or Markov given o_k .

5.2 Learning across Power Scenarios

For endpoint $e \in \{-, +\}$ let $J_e(\pi) = \mathbb{E}[C_\pi(\rho_e)]/C_{\text{ref},e}$, where $C_{\text{ref},e} > 0$ is the training-set mean for Fixed-4 under that endpoint. Let $J_V(\pi) = \Pr_\pi(T > D)$. For each configured budget, the objective is

$$\min_{\pi} \max_{e \in \{-, +\}} J_e(\pi) \quad \text{subject to} \quad J_V(\pi) \leq \epsilon. \quad (8)$$

The reference and all normalizers are frozen before validation. By Proposition 2 and the ratio property following it, the maximum also covers the interior of the chosen ρ interval for this same policy. This is robustness to power-accounting uncertainty; it does not imply robustness to arbitrary queue distributions. The chance constraint is a population target, not a guarantee for an individual job. If Slurm withholds all feasible allocations past D , every policy under this interface misses; no user-level controller can guarantee completion against unrestricted waits.

We optimize Equation 8 through

$$\min_{\theta} \max_{\substack{p \in \Delta_2 \\ \lambda \geq 0}} \sum_e p_e J_e(\pi_\theta) + \lambda [J_V(\pi_\theta) - \epsilon], \quad (9)$$

where Δ_2 is the probability simplex. At training iteration j ,

freeze p, λ for rollout and PPO updates; then update

$$p_e \leftarrow \frac{p_e \exp(\eta_p \hat{J}_e)}{\sum_f p_f \exp(\eta_p \hat{J}_f)},$$

$$\lambda \leftarrow [\lambda + \eta_\lambda (\hat{J}_V - \epsilon)]_+. \quad (10)$$

Budget-specific dual variables prevent an easy budget from hiding violations at a tight one. One actor is conditioned on the budget; models are trained separately per machine-workload panel, without a cross-workload transfer claim.

A shared two-layer, 128-unit ReLU action encoder and mean-pooled context produce four masked logits; the value network uses the same width. Three value heads estimate the remaining carbon cost at the two endpoints and the probability of an eventual deadline miss. For chunk k , the actor reward is

$$r_k = - \sum_e p_e \frac{C_k(\rho_e)}{C_{\text{ref},e}} - \lambda \mathbf{1}\{k = K - 1, T > D\}. \quad (11)$$

The carbon increment includes the entire allocation. All returns use $\gamma = 1$ and complete episodes. Their sum is exactly the negative Lagrangian cost up to the constant $\lambda\epsilon$, regardless of how many chunks the run needs. A fixed per-chunk discount would instead favor moving cost to later chunks. Value heads fit undiscounted Monte Carlo returns; the weighted cost advantage gives the PPO update. A missed deadline does not terminate training work or discard the job’s later carbon.

This construction reuses each execution to train both carbon heads and the miss head; it does not require a simulator run for every power parameter. The maximization is over expected episode cost, not a separate worst endpoint chosen at every chunk. PPO and the dual updates are numerical approximations, so validation and test must evaluate the attained carbon–deadline tradeoff.

5.3 Selection and Submission

Before evaluation, we freeze an empirical selection rule: among complete validation cohorts with observed miss rate at most ϵ , select the checkpoint minimizing the endpoint objective. Apply the same rule to baseline selection. If none qualifies, retain the complete candidate with the smallest miss rate as a labeled fallback. Empirical target attainment is distinct from a confidence bound supporting population feasibility; an inconclusive bound does not prove infeasibility. Test reports every configured budget and seed. The deployed action rule remains the same categorical sampling rule used for evaluation; changing to argmax requires a separate validation because it changes the policy.

The execution interface needs a profile, budget, visible queue snapshots, and a causal CI feed. It writes the selected node count and time limit into the submission and invokes `sbatch` (SchedMD 2026). The training process must save its checkpoint before allocation expiry; a wrapper cannot checkpoint a process after Slurm has killed it. Resubmission occurs only after a completed checkpoint and terminal job status are confirmed. A job-ID log prevents duplicate submission after wrapper restart. Missing inputs select a preconfigured fixed policy, whose outcomes remain part of evaluation.

This describes the intended interface; live deployment is not an established result of this draft.

6 Evaluation

Evidence status. The following four packages are reserved experiment designs and result slots. They specify the minimum evidence needed for the paper’s claims; they do not report completed experiments. The historical data were previously used in a train/validation split. We therefore use a retrospective chronological benchmark, disclosing prior use and fixing this revision’s train/validation/test intervals before execution. Fit on training and select on validation; do not revise methods using the new test outcomes. This protocol does not create a previously unseen holdout.

6.1 E1: Are the Work and Contention Scenarios Sound?

E1: inputs and scenario contract — experiment placeholder. Use Figure 12 of AMSP’s March 2024 revision (Chen et al. 2024): LLaMA-7B/13B/30B, BF16, 4096-token sequences, and fixed global batch. Its 32/128/512/1024-GPU points map to 4/16/64/128 eight-A800 nodes. Archive PDF hash, vector-bar extraction, units, and the simulation configuration. Audit input workloads, state initialization, work accounting, and independent wait probes. Scenario statistics and validation outcomes: **[E1: pending]**.

What the traces represent. A background job supplies a tuple of submission time, node demand, requested wall time, and occupied duration. These tuples define rigid, homogeneous-node workloads with no modeled network or I/O interference. Each source’s time order and joint width-duration structure remain intact; traces are not pooled. The nominal cluster has 128 nodes, matching the profile testbed’s size. This is a declared experimental capacity, not an estimate of either source cluster. Old durations specify scenario demands; they do not predict the old programs’ runtimes on A800s. Queue state and waits are regenerated under the declared scheduler. Conditional on identical tuples, initial state, and scheduler, changing hardware labels cannot change admission. That conditional property neither makes workloads hardware-independent nor establishes universal transfer.

Replay and validation. The main scheduler is a node-level conservative-backfill approximation, not Slurm itself. Rebuild background state from an empty state at the declared trace-prefix origin, excluding at least 28 days before scored arrivals and checking prefix sensitivity in development. Do not transplant historical starts or running allocations into the constructed cluster. Methods branch from the same background state; their requests then affect subsequent admissions. State continues across all chunks. Original observed waits are not ground truth for this constructed scenario. Verify scheduling mechanics on independently solvable cases and evaluate the predictor on held-out simulator probes; source-calendar tests preserve chronology and report censoring.

Execution inputs. Figure 12 reports tokens per GPU per second, z_n ; convert to updates per hour as $q_n =$

Input / check	Reserved evidence
Trace demand and burstiness	Pending: counts, widths, durations
Scenario contention	Pending: occupancy and wait tails
Published throughput input	AMSP Fig. 12; extraction archived
State and work conservation	Pending: scheduling/accounting checks
Prediction vs. probe wait	Pending: errors and coverage
Unmeasured execution costs	Declared overhead scenarios

Table 1: Input and validity results placeholder. Report both source workloads separately, including time zones, cleaning, clipping, and calendar blocks. Probe labels come from the simulator; they are not observed alternative requests on a production cluster.

$3600(8n)z_n/(1024 \cdot 4096)$. The 1024-sequence batch follows from the source’s stated micro-batch and accumulation settings. Extract only published points; 64- and 256-GPU points are absent. For each model, freeze $U_* = \lceil 192q_4 \rceil$ before scheduling experiments. This chooses a workload of approximately 192 useful-compute hours on four nodes; overhead and rounding determine the chunk count, rather than a convergence target. Use 600 seconds of total allocated overhead per chunk, split equally between initialization/restart and saving, as an explicit scenario. E4 varies this unmeasured cost. No checkpoint implementation, final-quality equivalence, or complete-node power measurement is claimed.

6.2 E2: Is Adaptation Better than Strong Alternatives?

E2: primary comparison — experiment placeholder. Evaluate paired initial arrivals for both source workloads and all three published profiles. Use EIA’s Texas ERCOT average consumption-based CO₂ intensity as the primary scenario (U.S. Energy Information Administration 2026). Set T_{ref} to the fastest fixed training mean TAT and D_{hi} to the training Fixed-4 95th-percentile TAT. Use $\beta_{\text{min}} = 1$, $\beta_{\text{max}} = D_{\text{hi}}/T_{\text{ref}}$, and $s \in \{0, 1/4, 1/2, 1\}$. This sets the budget span, not a feasibility guarantee. Freeze the grid, miss tolerance, power interval, seeds, and selection rule before test access. Compare ScaleDown with every fixed scale, a validation-fitted fixed mixture, plan-once control, and rollout planning. Main figure, carbon ratios, miss rates, TAT, and node-hours: **[E2: pending]**.

Baselines. Fixed-4/16/64/128 retain the same scale across all chunks and use the identical work, checkpoint, and final-request rules. *Best-Fixed* is selected on validation for each budget. *Fixed-Mix* samples one scale at the start of a run and keeps it: its four mixture weights solve the same endpoint objective and miss constraint using validation statistics. Mixture feasibility depends on the mixture of miss probabilities, not on an interpolated mean TAT. All four fixed points remain visible even if dominated.

For the planning baselines only, fit a request-conditioned boosted regressor for $\log(1 + w)$; chronological training residuals supply 32 equal-mass atoms of $\hat{F}_{k,n}$. Its causal

exposure estimate is

$$\hat{L}_{k,n} = n \mathbb{E}_{w \sim \hat{F}_{k,n}} \left[\int_{t_k+w}^{t_k+w+d_{k,n}} \hat{c}_{t_k}(s) ds \right]. \quad (12)$$

E1 reports probe errors and coverage to document this approximation; prediction accuracy is not an acceptance criterion for ScaleDown.

Rollout-MPC enumerates two upcoming scale choices and a constant-scale tail to completion, uses the auxiliary wait model and causal CI forecast, and executes only the first action before replanning. There are at most 4^3 candidate plans. Future queue observations are unavailable, so the rollout holds the current queue and wait-model calendar features fixed, updates request length and the forecast integration window, and samples conditional waits. Its carbon estimate adds past endpoint costs using released CI, retaining each chunk’s submission forecast for unreleased intervals, to the planned continuation. It selects the lowest worst-endpoint normalized carbon plan meeting the estimated miss tolerance, or the plan with the smallest miss estimate if none qualifies. This is a strong causal baseline with an explicit approximation, not a clairvoyant scheduler. *Plan-once* runs the identical planner only at the first submission and executes its selected scale sequence and constant tail without replanning. It receives the initial queue and forecast, allowing a planned within-run mix while removing feedback from later admissions. Both planning variants may tune their internal risk threshold on validation; test compares them under the same actual miss tolerance as every other method.

Paired analysis and selection. An episode is one initial arrival and complete run. Resample contiguous calendar blocks jointly across methods, with RL seeds as another replication level. Blocks must cover the history window and development dependence. Training fixes budgets and normalizers; validation fixes operating points. The primary comparator is the strongest validation-selected non-RL method meeting the empirical miss target at the same budget. Fixed-128 is not the automatic denominator. Confidence bounds are reported separately from this frozen empirical selection rule.

Report the endpoint ratio of *mean* carbon costs, mean and p95 TAT, miss rate, and allocated node-hours per panel; macro-averages use dimensionless panel ratios. Carbon includes all completed runs, including deadline misses. Episodes crossing a split must be purged or assigned entirely to one split, and any run not observed to completion is reported as censored, not silently dropped. Texas CI must have a documented source, release time, time zone, resolution, and missing-data rule. If CI and queue years do not overlap, their alignment is a labeled counterfactual pairing.

Result slot and inference. [E2: effect sizes, intervals, eligible budgets, and failed panels: pending]. An observed budgeted carbon reduction requires both frozen methods to meet the miss target on complete validation and test cohorts. This describes observed tradeoffs, not population feasibility. A stronger statistical claim also requires joint endpoint cost intervals and one-sided miss bounds to pass the declared criteria. Report inconclusive bounds, including for zero misses;

Method / variant	Carbon	Misses	Node-h
ScaleDown	—	—	—
Precommitted-RL	—	—	—
Rollout-MPC	—	—	—
Plan-once	—	—	—

Table 2: Feedback results placeholder at prespecified budgets. All dashes are pending E3 results. Use E2 normalization and paired intervals; compare carbon jointly with misses. Precommitted-RL changes its scale within a run but commits its choices before seeing subsequent outcomes.

nearby arrivals do not create independent calendar blocks. Gains over Fixed-128 alone establish neither adaptive nor RL value. An inspected former test period cannot be relabeled as untouched.

6.3 E3: Does Feedback Beat a Learned Sequence?

E3: matched feedback comparison — experiment placeholder. Reuse E2 ScaleDown, Plan-once, and Rollout-MPC. Train one additional *Precommitted-RL* policy with the same architecture, budget grid, seeds, arrival sampling, and episode-interaction budget. Before its first submission, it samples an entire scale sequence using only initial public queue/CI inputs, remaining planned work, and D minus planned allocation time. It receives no later queue observations, CI releases, or realized elapsed time. Evaluate at one prespecified budget per panel. Results: [E3: pending].

The learned-sequence comparison removes feedback while retaining the capacity to mix scales and optimize from completed episodes. Its remaining-time input is planned allocation time only, not a zero-wait assumption in the execution environment: all actual waits and costs enter training returns and evaluation. Archive each full plan before executing it. Neither policy needs an admission predictor. Matching interaction budgets prevents extra replay from explaining a feedback gain, although it cannot guarantee equal optimization quality. Plan-once versus MPC separately tests replanning within an explicit planner. Report single-chunk frequency: those runs offer no later decision. A fixed-scale win without a learned-sequence win supports mixing, not a feedback advantage; a win only over MPC could reflect its modeling error. Action frequencies alone do not establish either claim. Results: [E3: feedback increment: pending].

6.4 E4: Does the Conclusion Depend on the Power Model?

E4: robustness from existing logs — experiment placeholder. Freeze policies and recompute every E2 execution over the declared ρ interval, including both endpoints. Report pairwise break-even points, componentwise exposure differences, and the independent scale-error radius supported by Equation 6. Compare the robust policy with a single-endpoint-trained counterpart at one prespecified budget per panel. Reuse existing logs for parameter sensitivity; queue and forecast perturbations require fresh replay. Outcomes: [E4: pending].

Figure 2 reserves the sensitivity result. [E4: interval-wide advantage or the exact region of reversal: pending]. For $\Delta A = A_\pi - A_b$ and $\Delta B = B_\pi - B_b$, a possible break-

RESULT FIGURE PLACEHOLDER

E2: Two source workloads $\times$ three published AMSP profiles.
All fixed scales; Fixed-Mix; Plan-once; Rollout-MPC; ScaleDown.
Mean TAT versus modeled carbon at a declared ρ ; mark observed miss violations and label slider ticks.
Companion labels: worst-endpoint carbon ratio and deadline-miss rate.

Figure 1: Primary results placeholder. Every point completes the same prescribed work from paired initial arrivals. Report hours, modeled carbon normalized within each contention–profile panel, 95% paired block intervals, the power scenario, and the frozen budget grid. Mark observed miss-target violations explicitly. Fixed-Mix is an executable per-run randomized policy; connecting other points does not imply an evaluated intermediate policy. Report every slider tick; do not smooth reversals or select settings after test.

RESULT FIGURE PLACEHOLDER

E4: carbon advantage versus ρ .
Endpoint intervals; zero-gain line; break-even ρ .
Fixed policies and comparator throughout.
Small inset: endpoint-trained versus robust policy.

Figure 2: Power-sensitivity results placeholder. The parameter range is an assumed scenario, not a measured power range. Endpoint comparisons characterize the interval only under Equation 3. Plot uncertainty from paired execution samples separately from power-scenario variation.

even point is $\rho^* = -\Delta B / (\Delta A - \Delta B)$ when the denominator is nonzero. A point outside the declared interval is not an in-range reversal. Simultaneous one-sided endpoint intervals, obtained from the paired block resamples, support an interval-wide statistical statement; two point estimates alone do not. A nominal gain that reverses near the selected power assumption must be reported as model-sensitive.

Does the simulated environment determine the gain? At one prespecified budget and the 7B profile, reuse frozen full-policy and Rollout-MPC models in each source workload. Independently change (i) background widths to $\min(2n, 128)$, reporting the capped fraction, (ii) scheduling to strict FCFS, and (iii) total chunk overhead to 60 or 1800 seconds. Preserve physical budgets, workload, CI calendars, and all fitted parameters. These four additional settings require new replay, not new training or hardware measurements. Width and scheduler changes test learned queue relationships; E1 prediction accuracy alone cannot establish this robustness. Report carbon and misses together: a gain that disappears or violates the budget limits the claim. They test the selected contention and overhead scenarios, not every possible cluster. Results: **[E4: environment sensitivity: pending]**.

A compact reuse analysis reports E2 outcomes by held-out calendar block and CI variability. A constant-CI rescore of

the same logs removes timing-related carbon differences for those executions; it does not simulate policies reacting to constant CI. No cross-region, multi-user-adoption, or facility-wide experiment is needed for the scoped claim.

7 Limitations and Conclusion

Limitations. Results apply to the declared demand scenarios and A800 profiles, not production admissions on other hardware. Historical arrivals may encode user responses to their original systems; holding them fixed does not model reactions to changed waits. Scheduler settings, placement interference, and full-state checkpoint costs are unmeasured. An overhead sweep does not validate restart correctness. The policy still depends on learned queue relationships and fallible CI forecasts; hidden priorities or changed demand can invalidate them. A completion budget and empirical miss rate are not a production SLA.

Complete-node power is unmeasured. The sensitivity certificate assumes time-invariant scale coefficients in the declared model, excluding power changes from unmodeled phases or system behavior. Common factors cancel within panels, not across workloads. Job attribution does not establish what other users do with released capacity. We make no facility-wide, embodied-carbon, or adoption-impact claim. Training and profiling overhead lies outside per-run accounting and must be reported before claiming amortized benefit.

Conclusion. ScaleDown uses a completion-budget slider to control checkpoint-boundary resource requests without modifying the scheduler. It learns from complete outcomes, while an exposure decomposition makes unknown node power explicit. **[E2–E4: baseline-relative results, feasibility, and supported power interval: pending]**. A fixed-scale win establishes an opportunity; improvement over the learned precommitted sequence is needed to attribute gains to feedback. Claims about RL also require comparison with explicit planning.

References

- Achiam, J.; Held, D.; Tamar, A.; and Abbeel, P. 2017. Constrained Policy Optimization. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, 22–31.
- Chen, Q.; Hu, Q.; Wang, G.; Xiong, Y.; Huang, T.; Chen, X.; Gao, Y.; Yan, H.; Wen, Y.; Zhang, T.; and Sun, P. 2024. AMSP: Reducing Communication Overhead of ZeRO for Efficient LLM Training. *arXiv:2311.00257v2*, March 13.
- Ding, Q.; Zheng, P.; Kudari, S.; Venkataraman, S.; and Zhang, Z. 2023. Mirage: Towards Low-interruption Services on Batch GPU Clusters with Reinforcement Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '23*. New York, NY, USA: Association for Computing Machinery. ISBN 9798400701092.
- Fan, Y.; Lan, Z.; Childers, T.; Rich, P.; Allcock, W.; and Papka, M. E. 2021. Deep reinforcement agent for scheduling in HPC. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 807–816. IEEE.
- Hanafy, W. A.; Liang, Q.; Bashir, N.; Irwin, D. E.; and Shenoy, P. J. 2023. CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 7(3): 57:1–57:28.
- Nurmi, D. C.; Brevik, J.; and Wolski, R. 2008. QBETS: Queue bounds estimation from time series. In *Job Scheduling Strategies for Parallel Processing*, volume 4942, 76–101. Springer.
- Patterson, D.; Gonzalez, J.; Le, Q.; Liang, C.; Munguia, L.-M.; Rothchild, D.; So, D.; Texier, M.; and Dean, J. 2021. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*.
- SchedMD. 2026. Slurm Workload Manager: sbatch. Accessed September 15, 2026.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
- Shoeybi, M.; Patwary, M.; Puri, R.; LeGresley, P.; Casper, J.; and Catanzaro, B. 2019. Megatron-Lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*.
- Sukprasert, T.; Souza, A.; Bashir, N.; Irwin, D.; and Shenoy, P. J. 2024. On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud. In *Proceedings of the Nineteenth European Conference on Computer Systems*, 924–941. Association for Computing Machinery.
- U.S. Energy Information Administration. 2026. Hourly Electric Grid Monitor: ERCOT Historical Data. <https://www.eia.gov/electricity/gridmonitor/>. ERCO workbook, consumption-based hourly CO2 intensity. Accessed September 16, 2026.